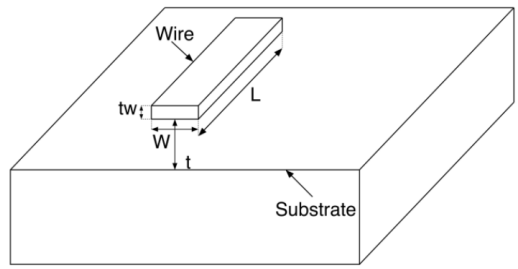
Wires

- Power wires carry DC signals, so the concept of wire delay does not exist, once they reached all blocks there is no need to care about delay (unlike signals and clocks).
- Metal wires are the type of wires mostly used.



Substrate Capacitance

$$C_{wire} = \frac{\epsilon A}{d} = \frac{\epsilon_o \epsilon_r W L}{t}$$

where ϵ_r is the relative permittivity of the insulator used.

- Which parameters in the capacitance equation we can use to decrease the wire capacitance?
  W: for sure we want to decrease it, but there is a minimum length for W defined by the technology's DRCs.
  L: for sure we want to decrease it, but it is not in our hand, it depends on the placement and routing that took place for the different blocks, once it is made the wire length is defined and cannot be decreased.
  t: defined by the technology and fab, so to change it we need to change the technology or the fab.
  ϵ_r: it is easy to say "let's use polyethylene for example since its ϵr is relatively small". However, it is easier to fabricate using silicon dioxide, so it is the insulator usually used.
- Which parameters are technology parameters, and which are design parameters?

$$C = \frac{\epsilon_o \epsilon_r W L}{t} = \left\{\frac{\epsilon_o \epsilon_r}{t}\right\}\{WL\} = C_a(WL)$$

  C_a: capacitance per unit area. This is the technology parameter (all wires in the same metal layer have the same C_a).
  WL: design parameter.

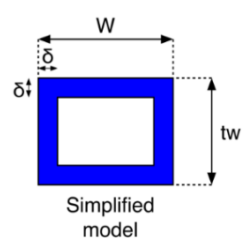
Resistance

$$R = \frac{\rho L}{A} = \frac{L}{\sigma W t_w}$$

- Which parameters are technology parameters, and which are design parameters?

$$R = \left\{\frac{1}{\sigma t_w}\right\}\left\{\frac{L}{W}\right\} = R_s\left\{\frac{L}{W}\right\}$$

  R_s: square/sheet resistance.
- The resistance of metal wires is no longer negligible in modern technologies due to:
  1. Skin effect.
  2. Scaling of wires.
- Skin effect: as technology increases, frequency increases. DC currents flow through the whole cross section of a wire. As the frequency increases, the skin depth in which the current moves decreases, which accordingly increases the wire's resistance. Simplified model:



$$A_{eff} = 2\delta W + 2\delta t_w - 4\delta^2$$

$$R_{eff} = \frac{L}{\sigma A_{eff}}$$

$$\delta = \sqrt{\frac{1}{\pi \mu f \sigma}}$$

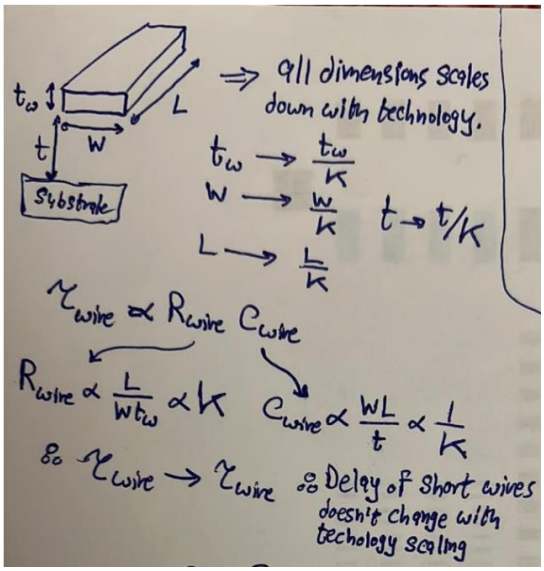
From the simplified model, skin effect takes place when:

$$\delta < \frac{1}{2}.\min(W, t_w)$$

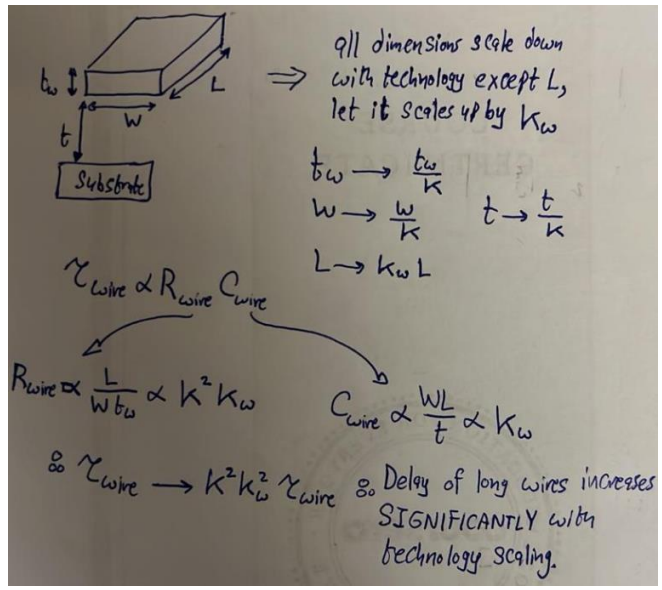
$$\therefore f_{min} = \frac{4}{\pi \mu \sigma . min^2(W, t_w)}$$

- Scaling of Wires: as technology advances, short wires maintain the same delay while global wires' delay increase significantly. Why?

Short Wires

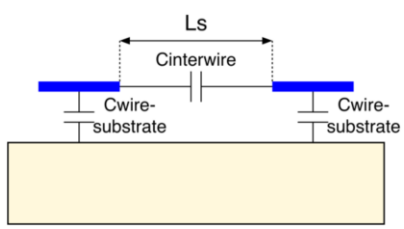


Long Wires



Interwire Capacitance

- It is a capacitance originated between two adjacent wires as shown below:



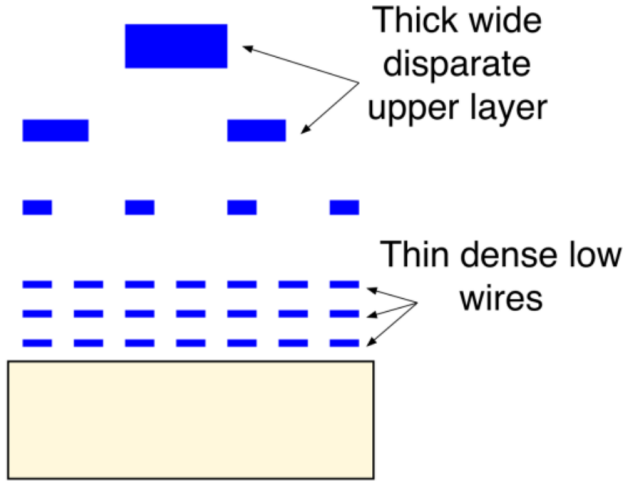
$$C_{interwire} = \frac{\epsilon t_w L}{L_{sep}}$$

L: common length between the two wires.

- Notice that R_{wire} kan nefsaha el thickness tezeed, while $C_{interwire}$ nefsaha el thickness ye2el. Thus, makansh asa7 qarar enena ma3 el technology advancements ne scale down el thickness bta3 l wires 3ashan keda $C_{interwire}$ hayzeed. Tab ne3mel eh 3ashan nerdy el tarafeen?

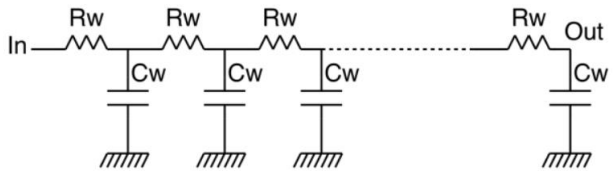
Hierarchical Routing

Kol ma hane3la fel metal layers, kol ma hanzawed el thickness beta3 el wires b7ees ne2alel el resistance beta3et-hom and hence use them for global routing. Bs keda el interwire capacitance mabenhom hatzeed? w 3ashan keda hanzawed kaman el separation mabenhom b7ees ne balance el denya. This way, wires in the upper metal layers have less resistance and hence can be used for global routing, while wires in lower metal layers are used for local routing. El shakl hayeb2a as shown below.

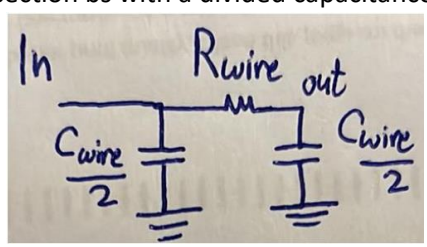


Modeling Wires in Delay Calculation

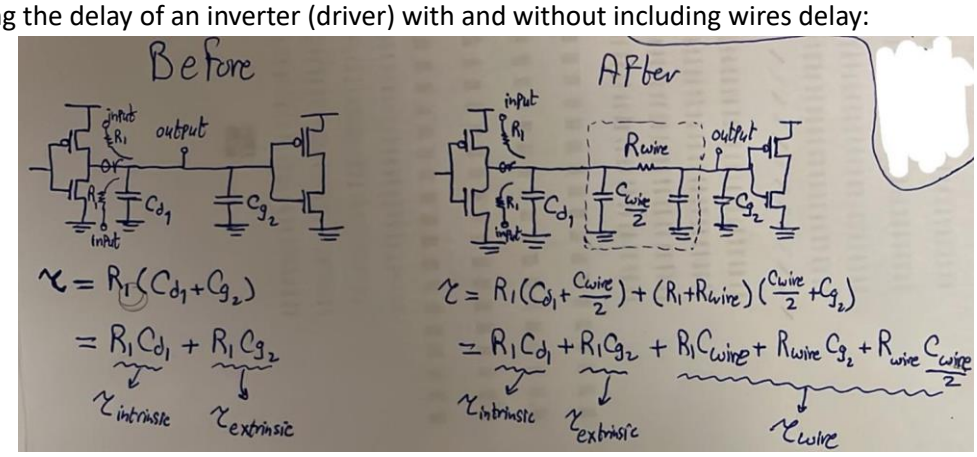
- El sa7 howa enena ne model el wire as an infinite number of pi sections as shown below:



- For simplicity, we will model it as a single pi section bs with a divided capacitance as shown below:



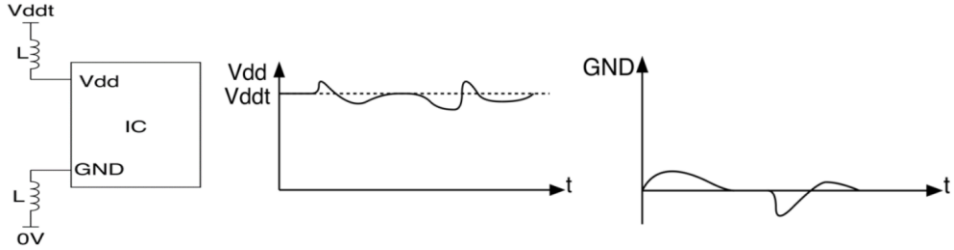
- Example of calculating the delay of an inverter (driver) with and without including wires delay:



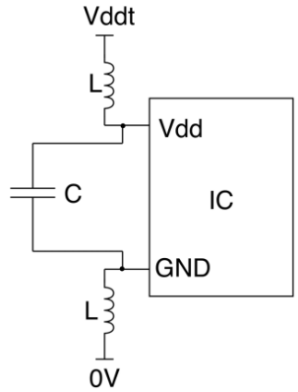
Notice enena bene7seb el time constant at the input of the second gate not the output of the first gate because this is what really matters.

Supply and Ground Bounce

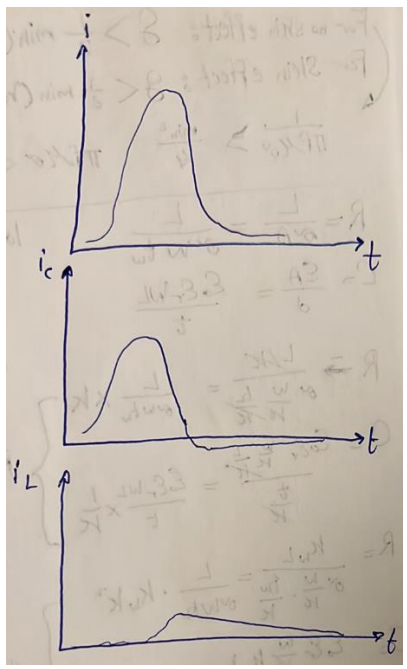
- V_{DD} and ground pins of a chip suffer from inductances to the supply and ground sources as shown below. Since these pins see large variation in current and $V_{inductance} = L\frac{dI}{dt}$, the voltage drop on the inductors can be large, which might cause V_{DD} to decrease and ground to increase, causing noise margins to decrease.



- One solution is to place a capacitor between the two pins as shown below. El capacitor haye3mel eh? wel signal 5arga metganena men el V_{DD} wel ground pins be frequency 3alya, el impedance beta3t el capacitor hateb2a soghayara w bel taly haya5od el signal de ye5aleha 3ando. Awel ma el frequency tesghar, terga3 el seka bta3t el inductors of low impedance tany, fa troo7 el signal safely lel V_{DDt} aw $0V$.



Eb2a erza3lo el kam rasma dool bardo:



Clocks

- Clock signal suffers from a large capacitance due to the capacitance seen from many flip flops AS WELL AS the capacitance of the clock wires themselves. The solution is to use a large buffer in the clock path.
- Lw el tool 3andaha el e5tyaar te5tar bera7et-ha mabeen positive w negative skew, hate5tar meen? most of the times hate5tar el negative skew mesh el positive skew. Leeh? la2en lw baseet 3la el equation beta3et el setup analysis wel hold analysis, hatla2y en el clock period T zahra fi el setup equation bs, mesh zahra fi el hold equation. El fekra b2a en el T de beteb2a akbar bekteer men qemet el skew δ, fa en e7na nedeef el δ de fi equation el setup sawa2 be tare2a tesa3do aw tare2a tedoro actually mesh betefre2 keteer la2enaha almost negligible bel nesba lel T. On the other hand b2a, el T mesh beteb2a mawgooda fi equation el hold, fa el times el da5la fi equation el hold beteb2a mesh kebera awy, fa el δ beyeb2a leha ta2seer actually fi el equation bta3t el hold. Fa 3ashan keda el tool bete5tar el δ such that enaha tesa3ed el hold equation, el howa enaha te5tar negative skew!

–	Clock Skew	Clock Jitter
Definition	Difference in arrival time of clock at different flip flops.	Deviations in timing of clock edges.
Source	Buffer mismatch, wire mismatch, load mismatch, temperature variations, power supply variations.	Phase noise of clock oscillator, temperature variations, power supply variations.
Effect	Positive skew may cause hold violations, while negative skew may cause setup violations.	May cause setup violation but no hold violations.
Properties	Spatial and Deterministic	Temporal and Stochastic

- If the data and clock are moving in the same direction —> positive skew.

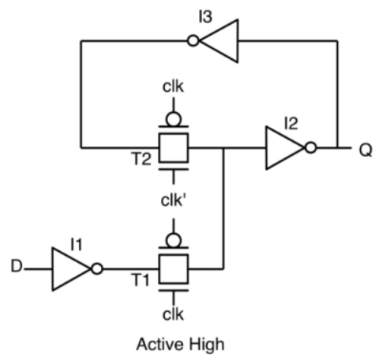
- Question: why doesn't jitter have any impact on hold time?
  Answer: because hold-time analysis is done on one clock edge, so even
  if jitter moves the active edge, it moves it for all registers simultaneously. Thus, regardless of the position of the edge, no new holdtime violations can occur. Skew had the potential to create new hold-time violations because it caused different registers to observe different clocks.

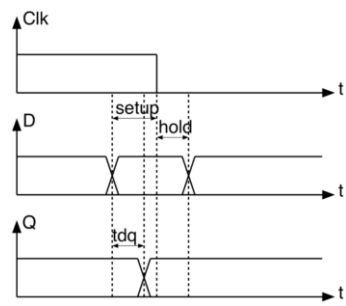
	Alpha Grid	Clock Tree
Advantages	• Design independent.	• Relatively small area and power. • Minimizes skew.
Disadvantages	• Large area and power. • Inevitable skew.	• Design dependent.

- Hybrid clock distribution is used in large designs. For example, when making a transceiver, the transmitter can be allocated a grid and the receiver can be allocated another grid.

Latches

Timing in Latches





$$t_{su} = t_{I1} + t_{T1} + t_{I2} + t_{I3}$$

$$t_h = t_{0-0\ overlap} - t_{I1} - t_{T1}$$

$$t_{dq} = t_{I1} + t_{T1} + t_{I2}$$

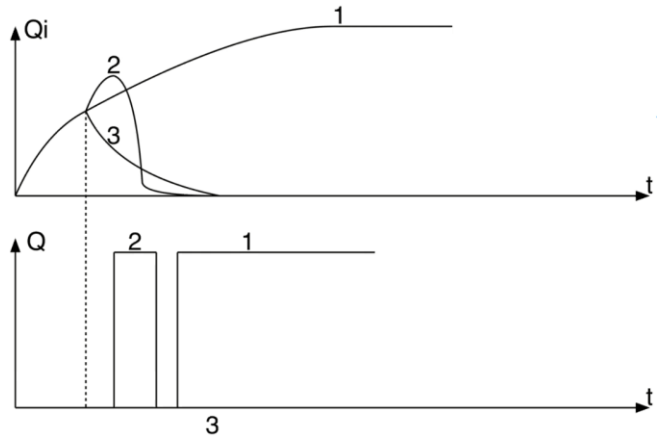
- VIP: fl flip flops el violations bt7sal 7awaleen el active edge, fel latches el violations bete7sal 3and nehayt el active level!!!!!
- El sa7 howa en kol stage ta7tarem nafsaha w t5ls fi wa2t $\frac{T_{clk}}{2}$ maximum. Tyb y3ny eh ta7tarem nafsaha? y3ny el wa2t el heya beta5do 3ashan tewasal el output beta3ha lel stage el ba3deha may3adesh $\frac{T_{clk}}{2}$. Tab eh el wa2t el ay stage beta5do? $t_{dq} + t_{CLB,max}$. Ay wa2t zeyada el stage de hata5do hayo3tabar borrowing, w bel taly $t_{borrow} = t_{dq} + t_{CLB,max} - \frac{T_{clk}}{2}$.
- For a correct operation of a latch loop, for each stage j:
  1. Calculate $t_{borrow_j} = t_{dq_j} + t_{CLB_j} - \frac{T_{clk}}{2}$.
  2. Verify that $\sum_{i=1}^{j} t_{borrow_i} \leq \frac{T_{clk}}{2} - t_{su}$
- Why don't we use latches in our design?
  1. Data has only $\frac{T_{clk}}{2}$ to propagate from one latch to another, while flip flops allows data to propagate in $T_{clk} - t_{su}$.
  2. Being level triggered, latches are vulnerable to glitches.
  3. Place and route tools find it harder to achieve timing closure in latch-based designs.

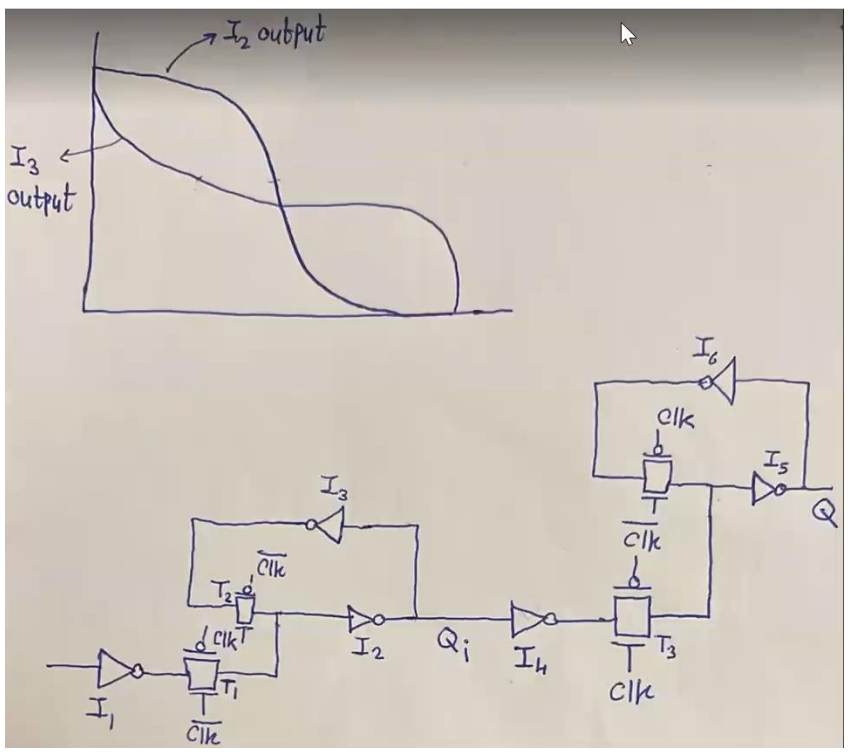
Clock Domain Crossing

- Two clocks are said to be in two different domains if they do not have a constant phase relationship. To have a constant phase relationship, the two clocks must:
    1. be generated from the same clock source.
    2. have frequencies that are multiple integers from each other.

How does a flip flop behave when it goes metastable?

- Metastability means that one of the following scenarios happened at the output of a flip flop:
    1. The output remained unchanged until the next clock cycle (when the input changes very near to the edge).
    2. The output glitched and returned back to its original value (when the input changes in the middle of the setup window).
    3. The output changed to the correct value but at a wrong time (when the input changes just after the setup window begins).





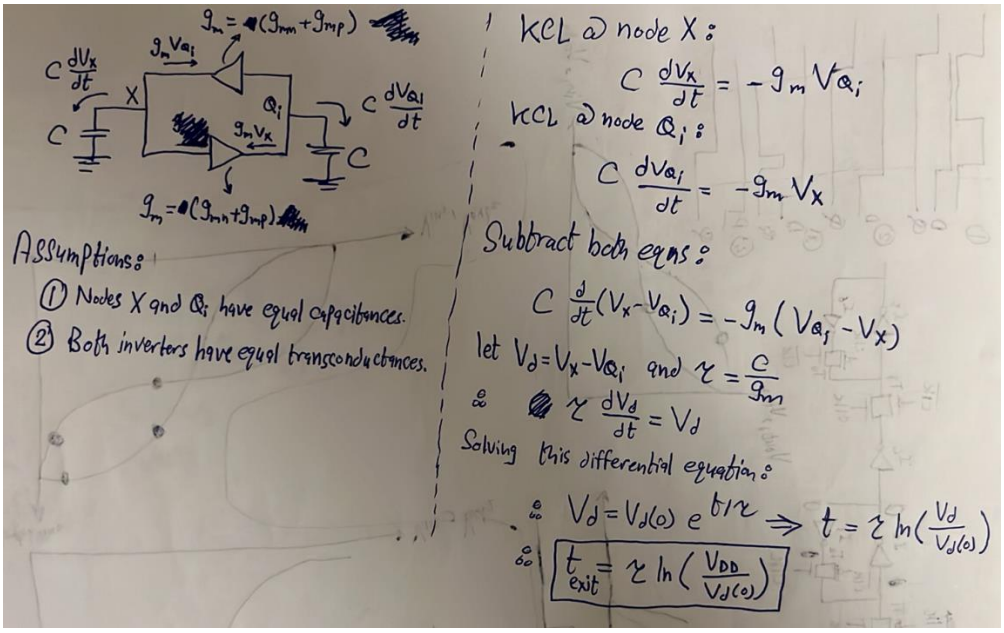
Lw nasy erga3 shoof video 1 fi lecture el CDC.

How often does metastability occur?

$$P(metastability) = \frac{T_m}{T_2}$$

$$MTBF|_{before\ synchronizer} = \frac{T_1 T_2}{T_m}$$

How long does it take a flip flop to get out of metastability?



$$P(still\ in\ metastability\ after\ time\ t \mid metastability\ occured) = e^{-\frac{t}{\tau}}$$

$$P(enters\ metastability\ \cap remains\ in\ metastability\ after\ time\ t) = \frac{T_m}{T_2} * e^{-\frac{t}{\tau}}$$

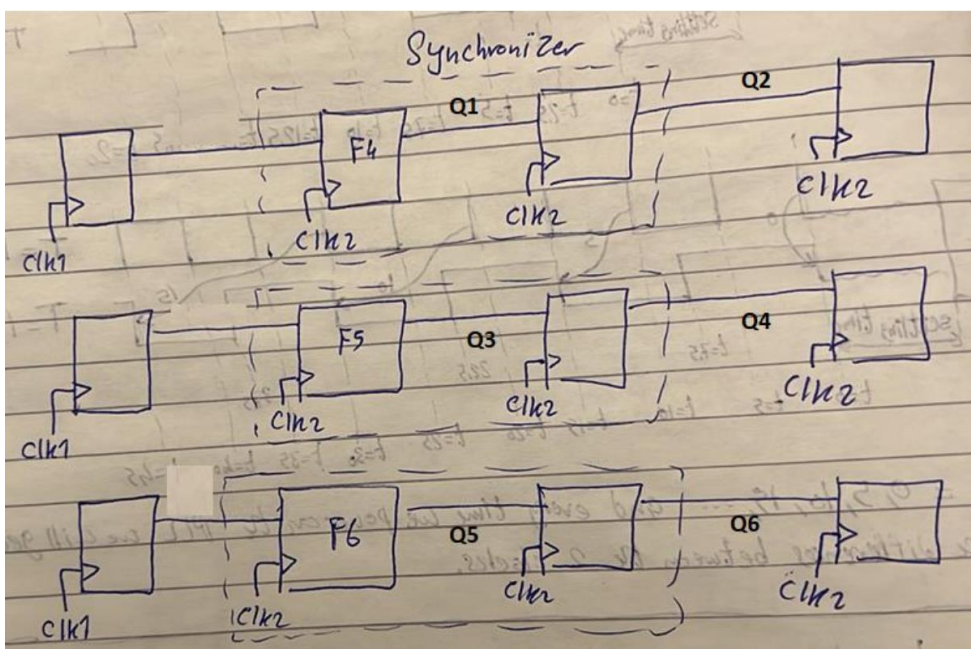
$$P(enters\ metastability\ \cap remains\ in\ metastability\ after\ time\ T_2) = \frac{T_m}{T_2} * e^{-T_2/\tau}$$

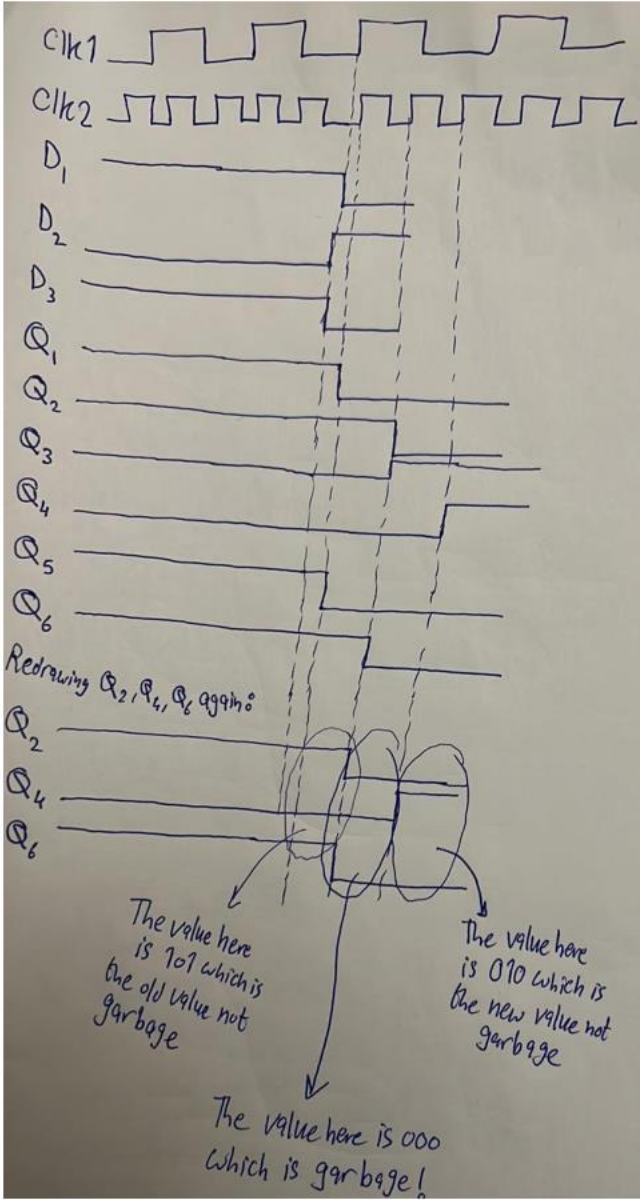
$$R(enters\ metastability\ \cap remains\ in\ metastability\ after\ time\ T_2) = \frac{T_m}{T_1 T_2} * e^{-T_2/\tau}$$

$$MTBF|_{after\ synchronizer} = \frac{T_1 T_2}{T_m} * e^{T_2/\tau}$$

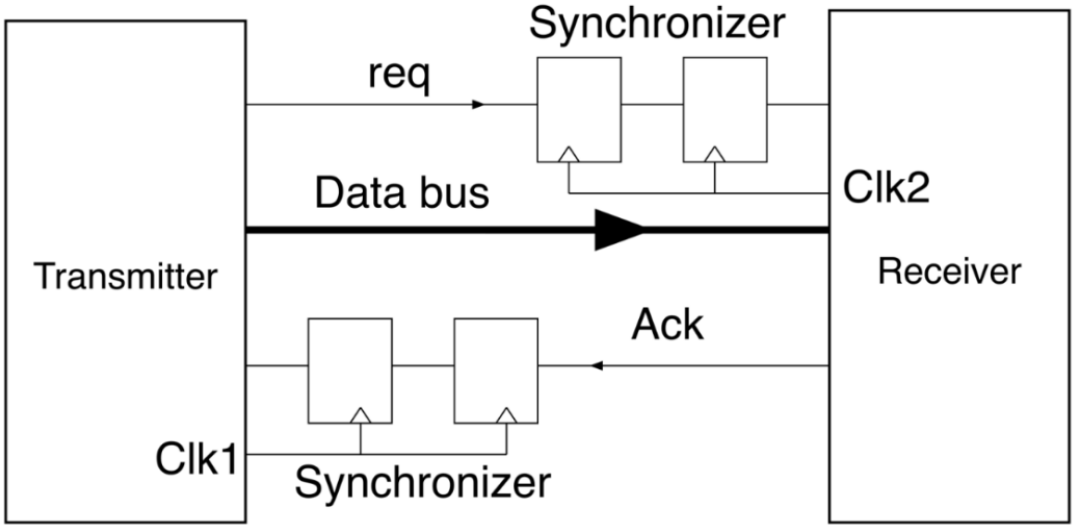
How to handle/solve metastability?

1. For a single bit, use a double flop synchronizer. Notice that it will have a one-cycle uncertainty.
2. For a gray counter, use multiple double flop synchronizers. Why can't we use multiple double flop with non-gray-counter buses? because garbage value might exist on the bus. Check the example below.





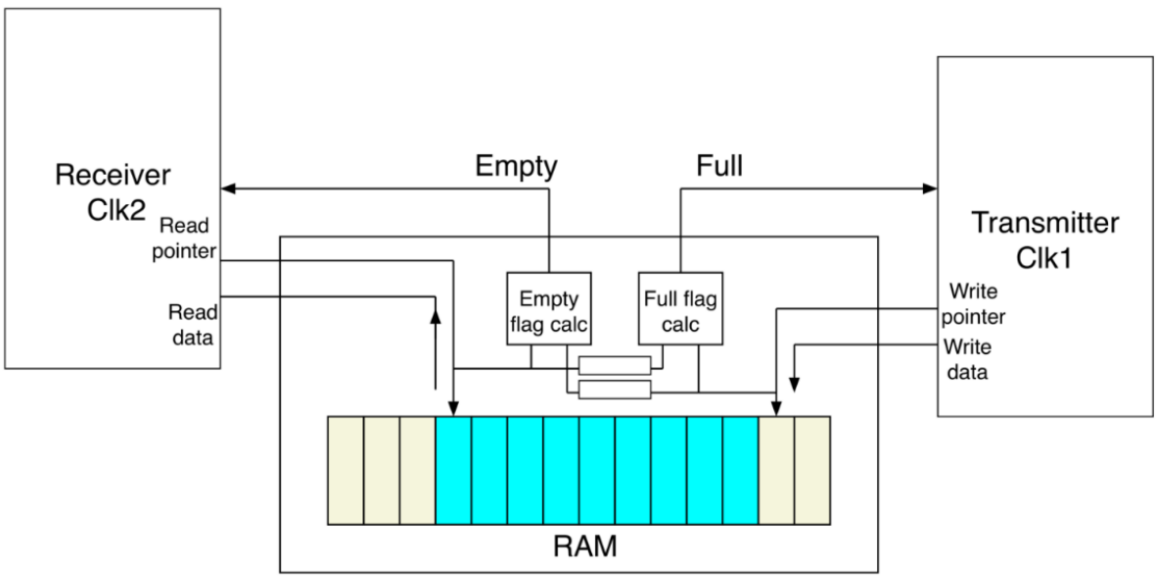
3. For non-gray-counter buses, handshaking synchronizers should be used.



The flow is as follows:

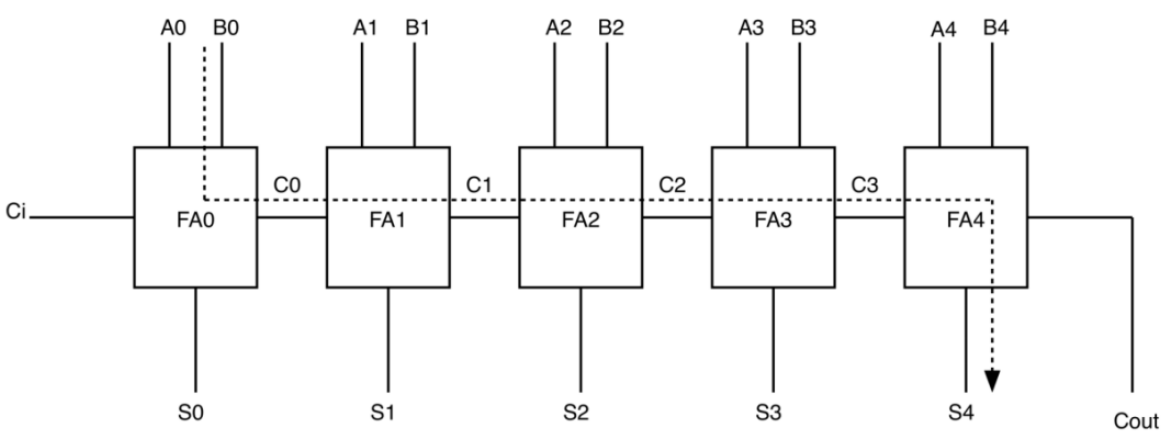
1. A data word is prepared on the bus synchronous with clk1. Simultaneously, req is raised on clk1.
2. Req is synchronized to the receiver clock clk2 using a double flop synchronizer. The receiver will either read the req on the second or third edge of the clock.
3. Because req and data were generated on the same clock, when req is synchronized at the receiver, it is safe to read the entire data bus. Thus, the receiver registers the data bus.
4. The receiver raises the "ack" flag on clk2, indicating the word has been read.
5. The ack flag is synchronized to the transmitter clock using a double flop synchronizer. The transmitter reads the ack flag on the second or third edge.
6. When the transmitter reads ack, it lowers the req signal, indicating it now understands the receiver has registered the data and it is now ready to send a new word.
7. The falling req signal is synchronized to the receiver. The receiver then lowers the ack signal, indicating it understands the transmitter might send a new word.
8. The falling ack signal is synchronized to the transmitter, indicating the receiver is ready to receive a new word.
9. The cycle can now restart, and a new word can be transmitted.

- Notice that too many clock cycles are wasted just to transmit one word of data from domain 1 to domain 2. As a result, using handshaking synchronization is only applicable when the transmitter and receiver communicate with each other occasionally.
- Another way to transfer data from a fast domain to a slow domain is using an asynchronous FIFO. One is shown below:



Adders

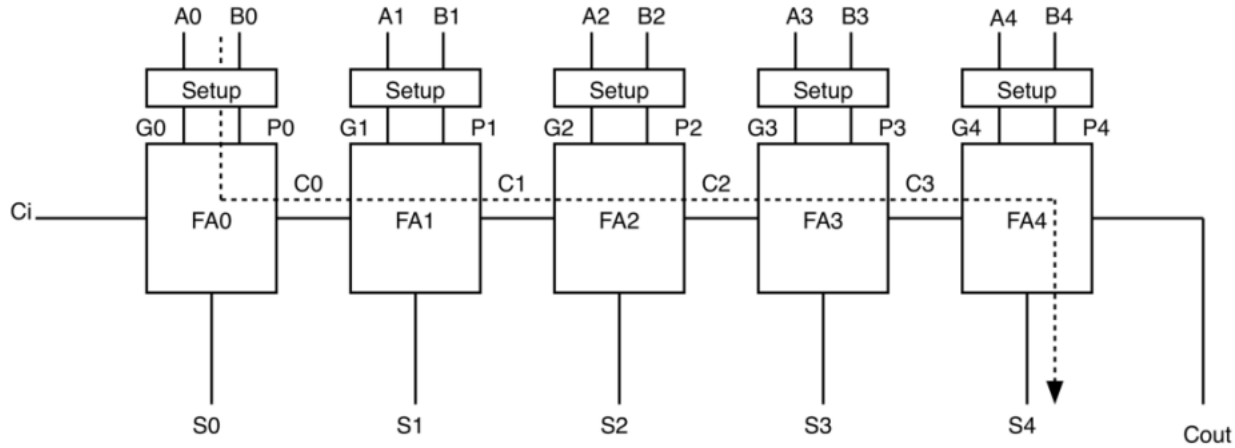
Ripple Carry Adder



$$T_{delay} = (N-1)T_{carry} + T_{sum}$$

- Delay grows linearly.

Ripple Carry Adder With Generate and Propagate Logic

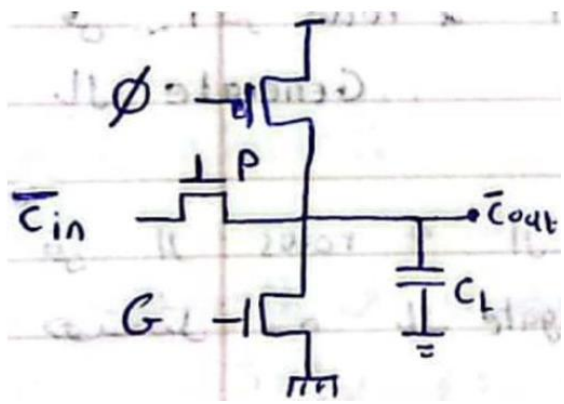


$$T_{delay} = T_{gp} + (N-1)T_{carry} + T_{sum}$$

where:

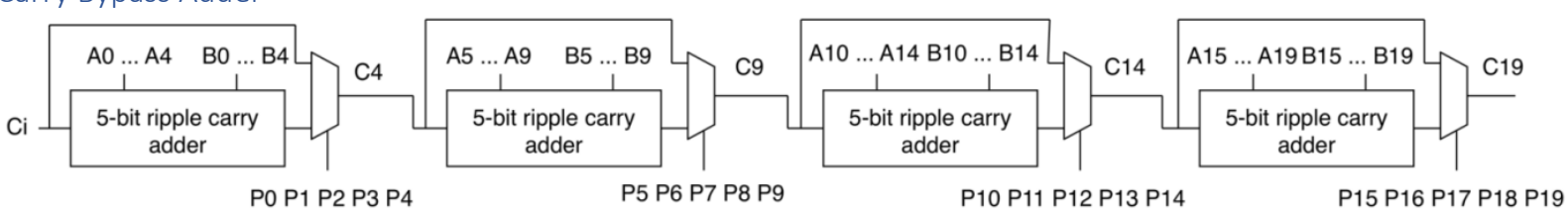
- T_{gp} is the delay to calculate G and P in parallel.
- $T_{sum} = P \wedge C_{in}$
- $T_{carry} = G + P.C_{in}$
- $G = AB$
- $P = A \wedge B$
- $D = A'B'$

Manchester Carry Chain



- Cascading results in an RC ladder, so the solution is to separate between each two stages using static buffers.

Carry Bypass Adder

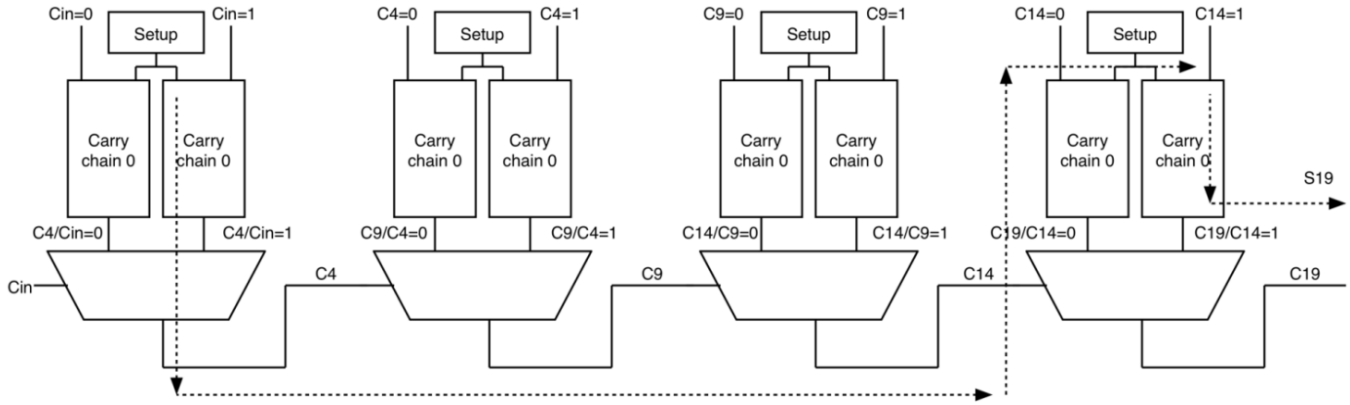


$$\boldsymbol{T_{add} = T_{gp} + KT_{carry} + (N/K-1)T_{mux} + (K-1)T_{carry} + T_{sum}}$$

- Delay growth is still linear but with a lower slope compared to the ripple carry adder.
- The optimal value of K is:

$$K = \sqrt{\frac{N}{2} \cdot \frac{T_{mux}}{T_{carry}}}$$

Carry Select Adder



- If all the stages have equal number of bits → same delay as carry bypass adder.
- If stage 1 has K bits, stage 2 has $K+1$ bits, and so on, then for P stages and using one chain in the first stage, the delay is:

$$T_{add} = T_{gp} + KT_{carry} + (P-2)T_{mux} + (K+P-2)T_{carry} + T_{sum}$$

$$N = (K) + (K+1) + (K+2) + \cdots + (K+P-1) = \frac{P(2K+P-1)}{2}$$

$$For\ P \gg K$$

$$N \approx \frac{P^2}{2} \rightarrow P = \sqrt{2N}$$

$$T_{add} = T_{gp} + KT_{carry} + \left(\sqrt{2N}-2\right)T_{mux} + \left(K+\sqrt{2N}-2\right)T_{carry} + T_{sum}$$

- Delay growth is square root.

Carry Lookahead Adder

- In this type of adder, we expand the expression of all carries and calculate each of them as shown below.

$$C_0 = G_0 + P_0\, C_{in}$$
$$C_1 = G_1 + P_1\, C_0$$
$$C_2 = G_2 + P_2\, C_1$$
$$C_3 = G_3 + P_3\, C_2$$
$$C_4 = G_4 + P_4\, C_3$$

$$C_0 = G_0 + P_0 C_{in}$$
$$C_1 = G_1 + P_1 G_0 + P_0 P_1 C_{in}$$
$$C_2 = G_2 + P_2 G_1 + P_1 P_2 G_0 + P_0 P_1 P_2 C_{in}$$
$$C_3 = G_3 + P_3 G_2 + P_2 P_3 G_1 + P_1 P_2 P_3 G_0 + P_0 P_1 P_2 P_3 C_{in}$$
$$C_4 = G_4 + P_4 G_3 + P_3 P_4 G_2 + P_2 P_3 P_4 G_1 + P_1 P_2 P_3 P_4 G_0 + P_0 P_1 P_2 P_3 P_4 C_{in}$$

- The delay growth is quadratic.

Group Generates and Propagates

- For the carry expressions below, each expression consists of group propagate, which is the last term without C_{in}, and group generate, which is the rest of the terms.

$$C_0 = G_0 + P_0 C_{in}$$
$$C_1 = G_1 + P_1 G_0 + P_0 P_1 C_{in}$$
$$C_2 = G_2 + P_2 G_1 + P_1 P_2 G_0 + P_0 P_1 P_2 C_{in}$$
$$C_3 = G_3 + P_3 G_2 + P_2 P_3 G_1 + P_1 P_2 P_3 G_0 + P_0 P_1 P_2 P_3 C_{in}$$
$$C_4 = G_4 + P_4 G_3 + P_3 P_4 G_2 + P_2 P_3 P_4 G_1 + P_1 P_2 P_3 P_4 G_0 + P_0 P_1 P_2 P_3 P_4 C_{in}$$

Group generate from bit x to bit y –symbol--> $G_{x:y}$. Example:

$$G_{2:4} = G_4 + G_3P_4 + G_2P_3P_4$$

Group propagate from bit x to bit y –symbol--> $P_{x:y}$. Example:

$$P_{2:4} = P_4P_3P_2$$

Thus:

$$C_j = G_{0:j} + P_{0:j}C_{in}$$

- Dot operator properties:
  - It can take groups of unequal lengths as inputs. Example: $(G_{1:3}, P_{1:3}) \circ (G_{4:9}, P_{4:9}) -> (G_{1:9}, P_{1:9})$.
  - It can operate on overlapping groups. Example: $(G_{1:3}, P_{1:3}) \circ (G_{2:9}, P_{2:9}) -> (G_{1:9}, P_{1:9})$.
  - Holes between the two input groups are not allowed. Example: $(G_{1:3}, P_{1:3}) \circ (G_{6:9}, P_{6:9})$ ✗.
  - It is not commutative.

$$(G_x, P_x) \circ (G_y, P_y) = (G_y + G_xP_y, P_yP_x)$$

where x and y are ranges.

Kogge Stone Adder

- Delwa2ty e7na 3ashan n2ool en el adder 5alaset, me7tageen kol el gy yeb2a et7asab:

$$S_0 = P_0 \oplus C_{in}$$

$$S_1 = P_1 \oplus C_0$$

$$S_2 = P_2 \oplus C_1$$

$$S_3 = P_3 \oplus C_2$$

$$C_3 = G_{0:3} + P_{0:3}C_{in}$$

$P_0 \rightarrow P_3$ w C_{in} kolohom m3ana from time 0. Keda fadel $C_0, C_1, C_2, G_{0:3}, P_{0:3}$. $C_0 \rightarrow C_2$ are given by:

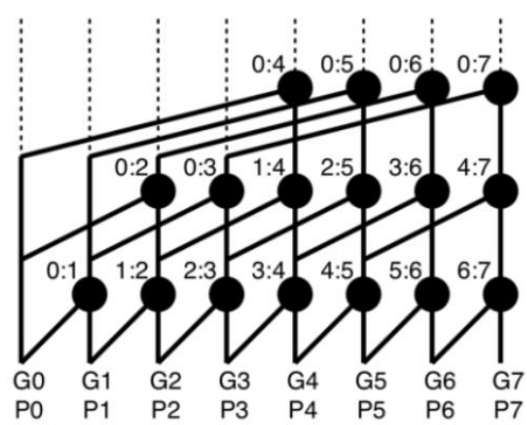
$$C_0 = G_{0:0} + P_{0:0}C_{in}$$

$$C_1 = G_{0:1} + P_{0:1}C_{in}$$

$$C_2 = G_{0:2} + P_{0:2}C_{in}$$

Yb2a men el a5er 3ashan el adder teb2a 5alaset e7na me7tageen $(G_{0:0}, P_{0:0})$, $(G_{0:1}, P_{0:1})$, $(G_{0:2}, P_{0:2})$, $(G_{0:3}, P_{0:3})$ yeb2o gahzeen. Once enohom kolohom ba2o m3ana, Hanroo7 7asbeen $C_0 \rightarrow C_3$ kolohom in parallel, then $S_0 \rightarrow S_3$ kolohom in parallel.

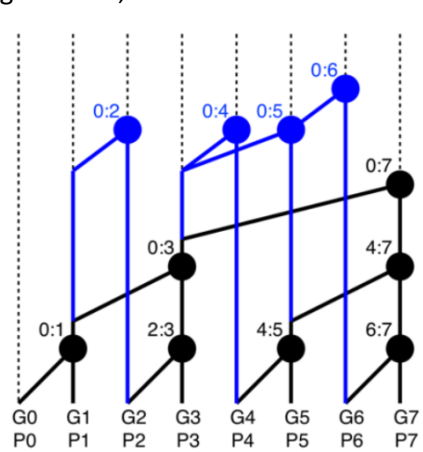
- Example of an 8-bit adder:

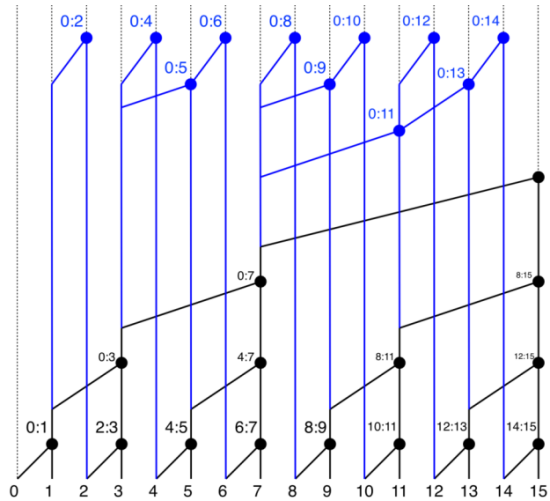


- The critical path has a delay of an N-bit Kogge Stone adder is $\lceil \log_2(N) \rceil * T_{dot}$.
- Delay growth is $log_2(N)$.

Brent Kung Adders

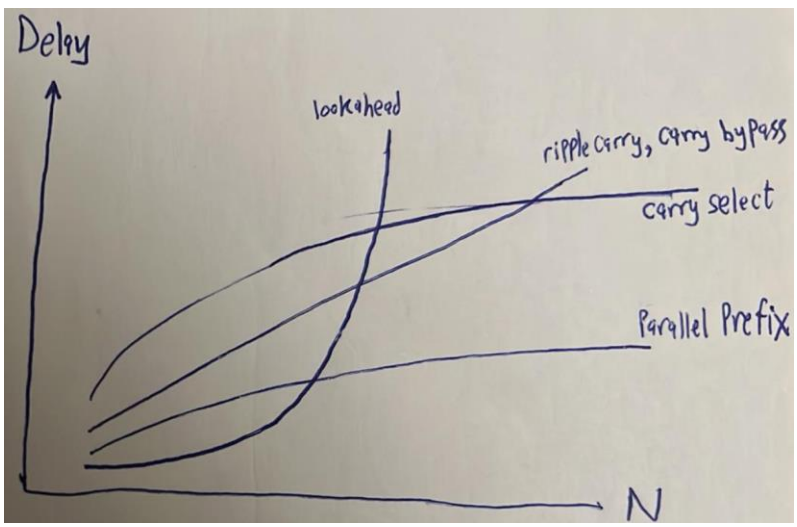
- Benkawen akbar range el awel, then nshoof eh el na2esna w ne3melo.





—	Kogge Stone Adder	Brent Kung Adder
Advantages	- Delay growth is $log_2(N)$. - Fanout of each dot operator is maximally 2 dot operators.	- The number of dot operators is less than that of a Kogge Stone adder and hence smaller area. - Less placement and routing headache than a Kogge Stone adder.
Disadvantages	- Area growth is more than $2N$. - Large headache for placement and routing tools.	- Delay growth is slightly larger than $log_2(N)$. - Fanout of each dot operator can be more than 2 dot operators.

Summary:



High Level Design (Not Totally Summarized)

- To convert a floating-point number to a fixed-point number stored in *n* bits on MATLAB:

$$a = rand;$$
$$a = a * (2^n - 1);$$
$$a = floor(a);$$

- SQNR is the ratio between the power of the signal to the power of the quantization noise as follows:

$$SQNR = \frac{(c_noisless)^2}{(c_noiseless - c_trunc * 2^p)^2}$$

USER DEFINED TYPES: ENUMERATIONS

```
type my_list is (oranges, apples, fifteen, seventy, John);
```

USER DEFINED TYPES: ARRAYS

~~type my_list is (oranges, apples, fifteen, seventy, John);~~

```
type register_file is array(array_lenght-1 downto 0) of
                    std_logic_vector(entry_size-1 downto 0);
```

USER DEFINED SUBTYPES

```
subtype short_vector is std_logic_vector(3 downto 0);
subtype eight_bit_int is integer range 0 to 255;
```

- Assuming there is an entity called *sixteen_bit_adder* as shown below:

```
entity  sixteen_bit_adder is
port (
input_a : in std_logic_vector(15 downto 0);
input_b :  in std_logic_vector(15 downto 0);
add_output : in std_logic_vector(16 downto 0)
);
end;
```

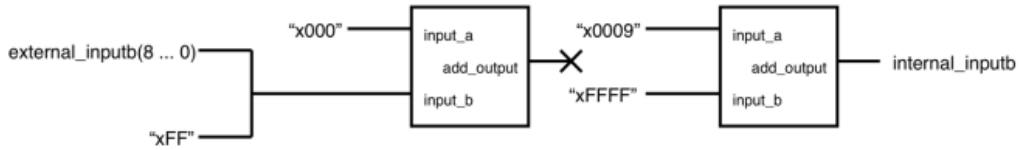
Instantiate an instance of this entity in an architecture (connect its ports to random signals).
Answer:

```
architecture hierarchical of layered_adder  is

component sixteen_bit_adder is
port(
input_a : in std_logic_vector(16-1 downto 0);
input_b :  in std_logic_vector(16-1 downto 0);
add_output : out std_logic_vector(17-1 downto 0)
);
end component;
begin
adder1 : sixteen_bit_adder
port map(
input_a => external_inputa,
input_b => external_inputb,
add_output => internal_inputa);
```

...

- For the same previous entity *sixteen_bit_adder,* instantiate the following instances:



Answer:

```
adder1 : sixteen_bit_adder
port map(
input_a => (others => '0'),
input_b => external_inputb(8 downto 0) & (others => '1'),
add_output => open);

adder2 : sixteen_bit_adder
port map(
input_a => conv_std_logic_vector(9,16),
input_b =>  (others => '1'),
add_output => internal_inputb);
```

- Selecting a specific architecture:

```
Add1 : entity work.sixteen_bit_add(behavioral1)
    port map (input_a => external_inputa,
        input_b => external_inputb,
        add_output => internal_inputa);
Add2: entity work.sixteen_bit_add(behavioral2)
    port map (input_a => external_inputc,
        input_b => external_inputd,
        add_output => internal_inputb);
```

- We can declare constants as follows:

```
architecture behavioral of bus_section_extraction is
constant extracted_part : integer := 10;
signal extracted_bus : std_logic_vector(extracted_part-1 downto 0);
begin
extracted_bus <= input_bus(extracted_part-1 downto 0);
end architecture;
```

- Generic maps can be declared as follows:

```
adder1 : n_bit_adder
generic map(
input_bus_width => 16
);
port map(
input_a => external_inputa,
input_b => external_inputb,
add_output => internal_inputa);
```

- Bos 3la el when-else wel with select t7t 3ashan lw gom maktobeen bs, mate7fazsh el syntax keda keda mesh hasta5demhom, hab2a akteb case gowa process :D

WHEN-ELSE MULTIPLEXING

```
sigout <= A when  S  = "00" else
    B when S= "01" else
    C when S = "10" else
    D;
```

WITH X SELECT MULTIPLEXING

```
With sel select
sigout <= A when "00" ,
    B when  "01",
    C when  "10",
    D when others;
```

- Some notes about variables:
  1. Variables are declared inside processes (between *process* and *begin*). Example:

```
process (A,B)
variable trial_var : integer := 0;
begin
C <= B;
D <= A;
E <= conv_std_logic_vector(trial_var,16);
end process;
```

  2. Variables keep their values from the last call (except for functions!!!! that is, variables do not keep their values between calls).
  3. Variables are assigned immediately.
- El beyetla3 men el sequence of if-else statements momken yeb2a mux kebeer OR a priority encoder (i.e. a cascade of 2x1 muxes). This depends on the conditions inside the if-else statements. If all the conditions are mutually exclusive, then one big mux is inferred. If they are not mutually exclusive, then a priority encoder is inferred.
- Ekteb two codes, wa7ed beye3mel flip flop with synchronous reset wel tany with asynchronous reset.

```
process(CLK)
begin
if CLK'event and CLK = '1' then
if reset = '1' then
Q <= '0';
else
Q <= D;
end if;
end if;
end process;
```

```
process(CLK, reset)
begin
if reset = '1' then
Q <= '0';
elseif CLK'event and CLK = '1' then
Q <= D;
end if;
end if;
end process;
```

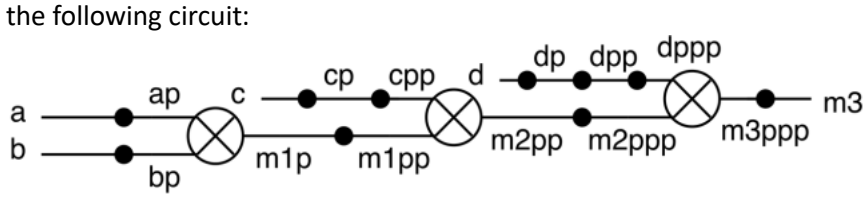
- Ekteb code for a general shift register.

```
architecture behavioral of shift_reg  is
type r_file is array(r_length-1 downto 0) of std_logic_vector(w-1 downto 0);
signal shift_reg : r_file;
begin

process(reset, CLK)
begin
if reset = '1' then
shift_out <= (others => '0');
elseif CLK'event and CLK = '1' then
for i in 0 to r_length loop-2
shift_reg(i+1)<shift_reg(i);
end loop;
shift_reg(0)<=shift_in;
shift_out<=shift_reg(r_length-1);
end if;
end process;

end architecture;
```

- Write a complete code for the following circuit:



Answer:

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_signed.all;

entity arith is
generic(data_width : integer := 16);
port(
a : in std_logic_vector(data_width-1 downto 0);
b : in std_logic_vector(data_width-1 downto 0);
c : in std_logic_vector(data_width-1 downto 0);
d : in std_logic_vector(data_width-1 downto 0);
m3 : out std_logic_vector(4*data_width-1 downto 0);
);

architecture bulk of arith is

signal ap, bp, cp, dp, cpp, dpp, dppp : std_logic_vector(data_width-1 downto 0);
signal m1p, m1pp : std_logic_vector(2*data_width-1 downto 0);
signal m2pp, m2ppp : std_logic_vector(3*data_width-1 downto 0);
signal m3ppp : std_logic_vector(4*data_width-1 downto 0);

begin

process(clk, rst) begin
if rst = '1' then
ap <= (others => '0'); bp <= (others => '0'); cp <= (others => '0');
cpp <= (others => '0'); m1pp <= (others => '0'); dp <= (others => '0');
dpp <= (others => '0'); dppp <= (others => '0'); m2ppp <= (others => '0');
m3 <= (others => '0');
elsif clk'event and clk = '1' then
ap <= a; bp <= b; cp <= c; cpp <= cp; m1pp <= m1p; dp <= d; dpp <= dp; dppp <= dpp; m2ppp <= m2pp; m3 <= m3ppp;
end if;
end process;

m1p <= ap * bp;
m2pp <= cpp * m1pp;
m3ppp <= dppp * m2ppp;

end architecture;
```

- Write a code for a ROM.

```
entity rom_block is
generic(
    address_bits : integer := 2;
    rom_word_width : integer := 4
);
port(
    clk : in std_logic;
    en : in std_logic;
    address_bus : in std_logic_vector(address_bits-1 downto 0);
    data_out : out std_logic_vector(rom_word_width-1 downto 0)
);
end entity;
architecture behavioral of rom_block is
type rom_type is array((2**address_bits)-1 downto 0) of
    std_logic_vector(rom_word_width-1 downto 0);
signal rom_contents : rom_type := (“0000”, “0001”, “1111”, “1110”);
signal read_data : std_logic_vector(rom_word_width-1 downto 0);
begin
    read_data <= rom_contents(conv_integer(address_bus));
    process(clk)
    begin
        if clk’event and clk = ‘1’ then
            if en=’1’ then
                data_out <= read_data;
            end if;
        end if;
    end process;
end architecture;
```

- Write a code for a single-port RAM.

```
entity ram is
    generic(
      add_bits : integer := 3;
      data_width : integer := 16);
    port(
      clk : in std_logic;
      rst : in std_logic;
      wen : in std_logic;
      ren : in std_logic;
      add : in std_logic_vector(add_bits-1 downto 0);
      data_in : in std_logic_vector(data_width-1 downto 0);
      data_out : out std_logic_vector(data_width-1 downto 0));
    end entity;

    architecture behav of ram is

    type ram_type array is ((2**add_bits - 1) downto 0) of std_logic_vector (data_width-1 downto 0);
    signal ram_contents : ram_type;

    begin

    process(clk, rst) begin
      if rst = '1' then
        ram_contents <= (others => '0');
        data_out <= (others => '0');
      elsif clk'event and clk = '1' then
        if wen = '1' then
          ram_contents( conv_integer(add) ) <= data_in;
        elsif ren = '1' then
          data_out <= ram_contents( conv_integer(add) );
        end if;
      end if;
    end process;

    end architecture;
```

- Check the cases below (note: fi 7alet eno wala bey read wala bey write mesh bey5osh fel code el t7t da aslan, fa data_out hat maintain a5er value leha, register y3ny).

IF I AM WRITING, WHAT DO I DO ABOUT THE DOUT PORT?

```
-- Read-first method, data_out takes the value of location from previous cycle
if we='1' then
    ram_memory(conv_integer(address_bus) <= data_in;
end if;
data_out <= ram_memory(conv_integer(address_bus);
-- Latched output. The data_out port preserves its last known value
if we='1' then
    ram_memory(conv_integer(address_bus) <= data_in;
else;
    data_out <= ram_memory(conv_integer(address_bus);
end if;
-- Write first. The value of the data_out port takes the new value
if we='1' then
    ram_memory(conv_integer(address_bus) <= data_in;
    data_out <= data_in;
else;
    data_out <= ram_memory(conv_integer(address_bus);
end if;
```

- Note: in case of multi-port memory, you MUST declare the memory as a *shared variable*, it cannot be a *signal.* Why? because the synthesis tool doesn't allow a signal to be assigned in more than one process.

- 3andena 4 types of wait statements:
  1. wait: not synthesizable.
  2. wait for: not synthesizable.
  3. wait until: most probably synthesizable. Example:

```
process
begin
wait until clk'event and clk='1';
Q <= D;
end process;
```

VIP: el example el t7t da bey create flip flops:

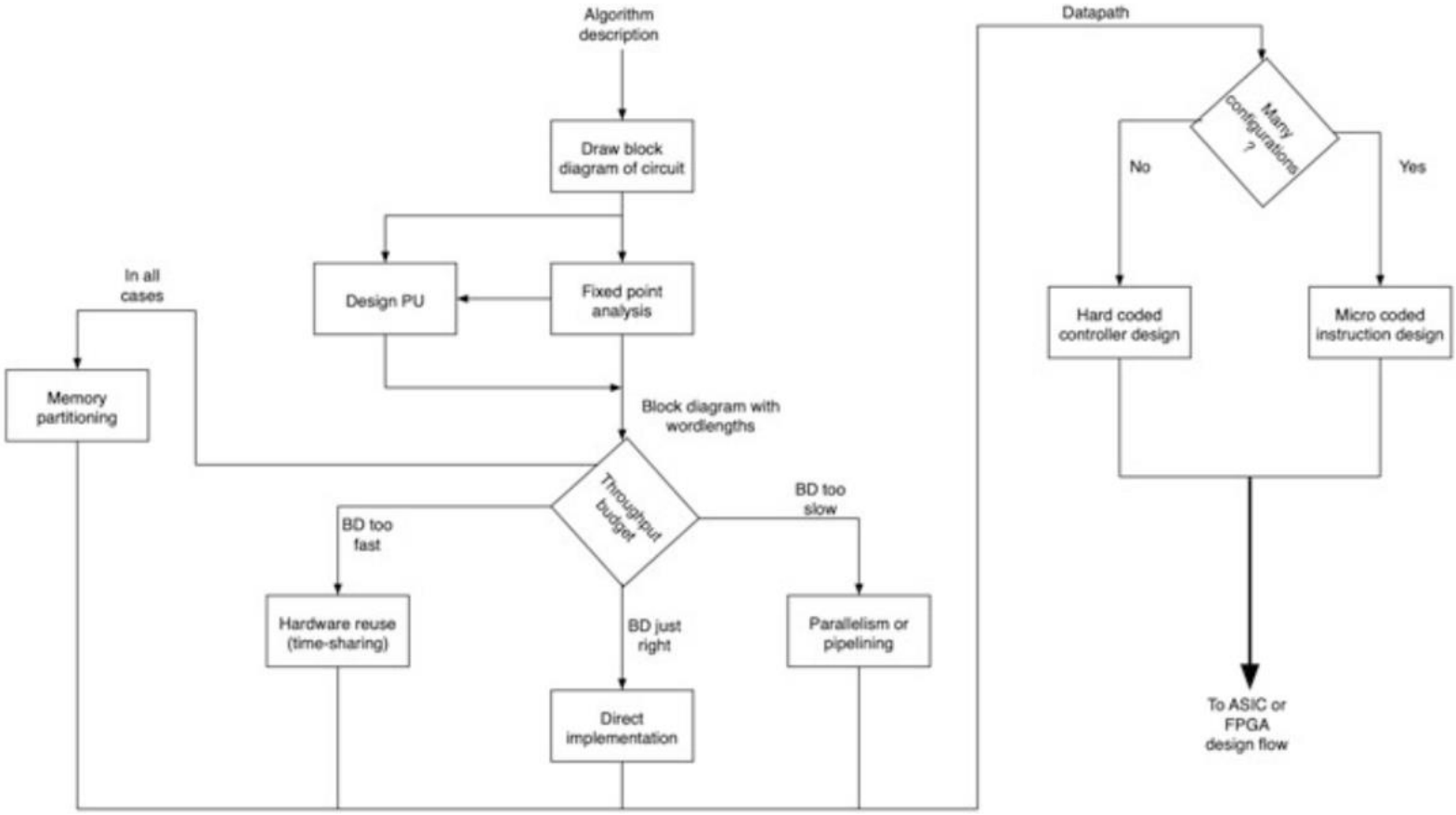
```
process
begin
Q <= D;
Q1 <= D1;
Q2 <= D2;
wait until loop_again = '1';
end process;
```

  4. wait on: de el sensitivity list.

Functions	Procedures
Declared in the declaration part of the architecture.	Declared in the declaration part of the architecture.
All ports are inputs without definition.	Ports can be inputs or outputs.
Must contain a return statement.	Doesn't contain a return statement.
Use variables only AND THESE VARIABLES DO NOT KEEP THEIR VALUES BETWEEN FUNCTION CALLS.	Can use variables and signals.
To be synthesizable, they must return a constant value.	To be synthesizable, they must return a constant value.

Example:
`function quad_calc(a,b,c,x : integer)`
`return std_logic_vector(16 downto 0) is`
`variable power1 : integer := 2;`
`variable power2 : integer := 1;`
`variable power3 : integer := 0;`
`variable return_value : integer;`
`begin`
`return_value := a*x**power1+b*x**power2+c*x**power3;`
`return conv_std_logic_vector(return_value,17);`
`end quad_calc;`
	Example:
	`procedure batch_calculate`
	`( a, b, c : in integer;`
	`signal d, e, f : out integer;`
	`signal g : inout integer) is`
	`begin`
	`d <= a*a;`
	`e <= b*b;`
	`f <= c*c;`
	`g <= g*g;`
	`end batch_calculate;`

- Why do we use 2's complement?
  1. It doesn't waste two numbers for representing a zero as in 1's complement and sign-magnitude representations.
  2. It is more arithmetic-friendly; we can mix positive and negative numbers without any pre-processing.
- Why do we use fixed-point?
  Although floating-point arithmetic is more accurate than fixed-point, we usually choose fixed-point over floating-point. Why? The main problem with floating-point is its arithmetic. Floating-point arithmetic requires much more power and processing compared to fixed-point arithmetic.
- Throwing the MSB causes failure in case of overflow, while throwing the LSB causes loss in accuracy in case of an odd result (take care of scaling, throwing the LSB translates to dividing the result by 2.



- Direct Implementation → number of PUs is equal to the number of operations.
- Parallelism → number of PUs is more than the number of operations.
- Resource sharing → number of PUs is less than the number of operations.
- Example:

WHY IS POWER IMPORTANT

- Power dictates how long we take between battery charges
  - Actually a combination of power, "speed", and usage
- A phone takes 7.56 microseconds to receive a packet
- Each packet has 1000 bits on average
- While receiving, the phone dissipates 1W
- It has a battery of 2000mAhr at a supply of 1V
- How many bits can the phone receive before needing a recharge?

Answer:

$$\text{Battery Life} = \frac{2000\,mA\,hr \times 1V}{1\,W}$$
$$= 2\,hr$$

$$* \text{ bits/sec} = \frac{1000}{7.56 \times 10^{-6}} = 132.275 \text{ M bit/sec}$$

$$* \text{ bits} = 132.275 \times 10^{6} * 2 \times 60 \times 60$$
$$\cong 9.524 \times 10^{11} \text{ bits.}$$